PROJECT SPARK | LEVEL 14

LED Protection & Current Limiting

Creation guide | 5 V DC | 4 objectives | Replaceable resistor

1 | Mission and topology

Player replaces a 150 ohm current-limiting resistor with 330 ohm, ensures correct LED polarity and connects a fixed 470 ohm branch in parallel. Switch OFF before replacement.

Circuit: 5 V (+) -> switch -> (A) R_LIMIT -> LED1 anode -> LED1 cathode -> GND; (B) R_REF 470 ohm -> GND. Both branches begin at switched +5 V and end at common ground.

2 | Prefab content

Clone the established level-layout structure into **Level_14_Layout**; link to Level_14.asset. Keep PowerSupply_01 and switch. Add an LED with anode/cathode terminal objects, a fixed 470 ohm resistor, and an empty replaceable resistor workstation. Ensure R_LIMIT is in series with the LED, never bypassed.

Initial installed resistor: 150 ohm; correct shelf replacement: 330 ohm. Use the existing shelf/workstation system. Give this workstation its actual registered SlotId (do not assume Register_Slot_2 unless verified).

3 | Wiring checklist

1. Battery_positive -> switch input.
2. Switch output -> R_LIMIT input.
3. R_LIMIT output -> LED1 anode.
4. LED1 cathode -> Battery_negative.
5. Switch output -> 470 ohm reference resistor input.
6. 470 ohm reference resistor output -> Battery_negative.
7. LCD A -> R_LIMIT input; LCD B -> R_LIMIT output (display ~3.00 V).

4 | Bindings and workstation bridge

Fixed component IDs: PowerSupply_01, LED1, Resistor_470_Ω (if using that object name). LED terminals should be mapped as LED1_Anode and LED1_Cathode. Battery_positive / Battery_negative remain as exported.

Runtime declarations on primary SparkLevelSceneBindings: runtimeComponentIds = R_LIMIT_Component; runtimeTerminalIds = R_LIMIT_Input, R_LIMIT_Output.

Bridge rule: component=R_LIMIT_Component, input=R_LIMIT_Input, output=R_LIMIT_Output, electrical type SparkResistor; terminal suffixes _Input / _Output. Set SlotId to the actual workstation slot ID registered by SparkObjectShelf.

5 | Four imported objectives

1. **LED Conducting** - ComponentConducting (type 4), LED1. NOTE: checks conduction, not LED animation or illumination.
2. **Protected LED Branch Current** - CurrentRange (type 6), R_LIMIT_Component, 0.0080 to 0.0105 A.
3. **Correct Resistor Voltage** - VoltageRange (type 5), R_LIMIT_Input to R_LIMIT_Output, 2.70 to 3.30 V.

4. **Safe Total Consumption** - CurrentRange (type 6), PowerSupply_01, 0.0180 to 0.0210 A.

6 | Expected measurements (illustrative diode approximation)

Assuming 2.00 V forward LED drop: I_LED = (5 - 2)/330 = **9.09 mA**; V_RLIMIT = **3.00 V**; I_470 = 5/470 = **10.64 mA**; source total = **19.73 mA**. A 150 ohm limiter predicts 20 mA in the LED branch and 30.64 mA total.

Calibration required: These are not guaranteed Unity solver values. Confirm the LED electrical model produces appropriate forward voltage and conducting state.

7 | Import and verification

Back up the original Level_14.asset. Select it in Unity Project view and run the existing editable-TXT importer on Level_14_LED_Protection_Import.txt. This retains the uploaded asset path and GUID.

In Edit Mode check Level_14_Layout primary bindings, new LED references, runtime declarations and workstation SlotId. In Play Mode install 330 ohm, run bridge Synchronize + Report Now and confirm all aliases VALID.

Inspect electrical audit: LED and fixed reference resistor conduct; LED orientation correct; source ~19.73 mA; limiter ~9.09 mA and ~3 V. Check level goals 4/4 and observe LED glow.

Important implementation gate

The uploaded file does not define an LED component type, LED target enum, or known LED terminal names. This package proposes **LED1** as an electrical component and uses the proven ComponentConducting target enum (4). If SparkLED does not derive from SparkElectricalComponent / bind as a component, objective #1 must be adapted after reviewing the project C# scripts. Avoid claiming this untested asset is immediately playable.